Universidad del Valle de Guatemala

Facultad de Ingeniería

Departamento de Ciencias de la Computación

Construcción de Compiladores

Proyecto Fase 3

Generación de Código MIPS para Compiscript

Informe técnico de arquitectura, implementación y validación

Compiler Lab

Repositorio: `https://github.com/Emadlgg/compiler-toolkit`

Autor

Osman Emanuel de León García – Carné 23428

Guatemala, noviembre de 2026

Índice

1. Resumen ejecutivo **2**

2. Objetivos y alcance **2**

3. Arquitectura y flujo de compilación **2**

4. Secuencias de llamada y retorno **3**

5. Asignación y preservación de registros **3**

6. Generación de MIPS y soporte en tiempo de ejecución **4**

7. Extensión: bibliotecas compiladas e importaciones **4**

8. Validación y pruebas **4**

9. Limitaciones y consideraciones **5**

10. Conclusiones **5**

Referencias técnicas **5**

1 Resumen ejecutivo

La tercera fase del proyecto consiste en transformar programas escritos en Compiscript, lenguaje basado en un subconjunto de TypeScript, en código ensamblador MIPS32 ejecutable en el simulador MARS. La solución integra análisis léxico y sintáctico con ANTLR, análisis semántico, generación de código de tres direcciones (TAC), traducción a MIPS y una interfaz de desarrollo para compilar, inspeccionar y ejecutar programas.

La implementación documentada incorpora registros de activación, paso de parámetros, conservación del contexto de ejecución y asignación de registros mediante `getReg()`. Además, ofrece un mecanismo adicional de bibliotecas de clases compiladas (`.cplobj`), importación y herencia entre archivos.

En las pruebas realizadas en el equipo de desarrollo se obtuvo un resultado de **166 pruebas automatizadas aprobadas, sin fallos**, con una advertencia sobre una entrada duplicada en un archivo ZIP de una prueba de manipulación de bibliotecas. Además, se ejecutaron desde el IDE los ejemplos de MIPS utilizados en la demostración. Las pruebas permiten comprobar el comportamiento de los casos cubiertos; la revisión de los commits individuales y la presentación en vivo forman parte de la entrega.

2 Objetivos y alcance

Objetivo general. Implementar un generador de código MIPS funcional a partir del código intermedio de Compiscript y verificar su ejecución en un simulador.

Objetivos específicos.

- Traducir instrucciones TAC a operaciones y estructuras de control MIPS32.
- Implementar llamadas y retornos de procedimientos, incluyendo argumentos, dirección de retorno y restauración del contexto.
- Administrar registros temporales y preservados, recurriendo a memoria cuando sea necesario.
- Integrar generación y ejecución al IDE y disponer de pruebas exitosas y fallidas.

El informe se limita a la **Fase 3 (generación de código)**. El intérprete SQL corresponde a otro enunciado y no forma parte de esta evaluación técnica.

3 Arquitectura y flujo de compilación

```
Archivo .cps ⟶ ANTLR (lexer/parser) ⟶ semántica
    ⟶ TAC ⟶ MIPSGenerator ⟶ archivo .asm
        ⟶ MARS ⟶ salida / diagnóstico
```

El proceso comienza con el análisis del archivo fuente. Una vez comprobadas las reglas semánticas, se genera el TAC que sirve como entrada para la siguiente etapa. El generador traduce ese código intermedio a instrucciones de MIPS y produce un archivo `.asm`. El simulador MARS permite ejecutar y observar la salida del programa.

Componente	Responsabilidad
`compiscript.py`	Punto de entrada de compilación y opciones para MIPS y bibliotecas.
`compiscript/mips/generator.py`	Traducción del TAC y emisión de instrucciones MIPS.
`compiscript/mips/frames.py`	Convenciones de activación y organización de la pila.
`compiscript/mips/registers.py`	Administración de registros mediante `RegisterAllocator.getReg`.
`gui/app.py`	Integración de compilación, visualización y ejecución desde el IDE.
`tests/mips/`	Casos de ejecución y comparación de resultados.

4 Secuencias de llamada y retorno

La arquitectura utiliza una pila descendente controlada por `$sp` y marcos de activación referidos por `$fp`. La distribución descrita en la documentación del proyecto es:

Desplazamiento respecto a `$fp`	Contenido
`+0`	Valor de retorno
`+4`	Dirección de retorno `$ra`
`+8`	`$fp` del llamador
`+12`	Enlace estático para funciones anidadas
`+16` y siguientes	Parámetros, variables locales y temporales

Antes de la llamada, el llamador reserva espacio para el marco, almacena el contexto requerido y coloca los argumentos en posiciones determinadas; a continuación utiliza `jal`. El procedimiento llamado establece `$fp` y guarda `$ra`. En el retorno, deposita el resultado en el marco y en `$v0`, restaura los registros y la pila, y transfiere el control mediante `jr $ra`. El enlace estático facilita el acceso a variables de ámbitos anidados.

5 Asignación y preservación de registros

La implementación distingue registros temporales y registros preservados. Los registros `$t0-$t7` se usan para operandos y cálculos transitorios dentro de una instrucción TAC; se solicitan mediante `getReg()` y se liberan al finalizar. Los registros `$s0-$s7` alojan variables locales y parámetros con mayor beneficio de permanencia durante una función.

La planificación de los registros preservados cuenta los usos de variables y pondera las apariciones en bucles; selecciona hasta ocho candidatas por función. No se incluyen las variables globales ni aquellas que deban permanecer accesibles desde funciones anidadas. Los registros `$s` utilizados se guardan al entrar y se restauran antes del retorno.

Cuando no existen registros libres, el asignador puede desalojar un valor mediante una política *round-robin* y un mecanismo de *spill*. Los temporales TAC y las variables sin registro asignado mantienen ubicaciones en memoria. Así se cubre el requisito de asignación de registros o utilización de la pila.

6 Generación de MIPS y soporte en tiempo de ejecución

El generador produce ensamblador MIPS32 a partir del TAC. El soporte de ejecución documentado contempla impresión de enteros, booleanos y cadenas; operaciones de concatenación y comparación de cadenas; y diagnósticos para división por cero, referencias nulas, índices fuera de rango y tamaños de arreglo inválidos.

La compilación desde consola se realiza con:

```
python compiscript.py programa.cps --mips --output build/programa.asm
java -jar tools/Mars.jar nc build/programa.asm
```

Desde el IDE, **BUILD MIPS** genera el ensamblador y **RUN MIPS** lo ejecuta mediante MARS. Las pestañas *MIPS CPS* y *Consola MIPS* permiten inspeccionar el código y su salida. Se requiere Java y `tools/Mars.jar`, o configurar la variable `MARS_JAR`.

7 Extensión: bibliotecas compiladas e importaciones

Además de los requisitos de la Fase 3, se incorporó la posibilidad de exportar clases, compilarlas como bibliotecas `.cplobj` y utilizarlas desde otros archivos. La biblioteca conserva código compilado e información de interfaz; la ruta de importación se interpreta respecto al archivo que importa.

```
# Compilar una biblioteca
python compiscript.py Animal.cps --build-library --output build/Animal.cplobj
```

```
// En otro archivo .cps
import { Animal } from "./build/Animal.cplobj";
export class Perro : Animal {
    // Miembros propios de la clase derivada
}
```

Esta funcionalidad permite organizar clases en distintos archivos y reutilizarlas sin volver a copiar su código fuente. Las dependencias registran hashes SHA-256, por lo que un cambio en una biblioteca padre exige recompilar las bibliotecas hijas. La documentación establece dos restricciones: las bibliotecas compiladas no admiten variables globales y, durante su construcción, sus importaciones deben apuntar a otras bibliotecas `.cplobj`, no a fuentes `.cps`.

8 Validación y pruebas

La batería MIPS compila programas Compiscript, los ejecuta en MARS y compara la salida obtenida con archivos de referencia `.out`. También incluye casos erróneos cuya salida o diagnóstico debe coincidir con los mensajes esperados en `.err`. Las rutas documentadas incluyen `tests/mips/cases/ok`, `tests/mips/cases/error` y `tests/codegen/ok`.

```
python tests/mips/run_tests.py -v
python -m pytest tests -q --tb=short
```

El botón **TEST MIPS** ejecuta la batería desde el IDE y presenta los resultados en *Pruebas CPS*; **TEST CPS** corresponde a las pruebas del código intermedio.

Evidencia reportada	Resultado / interpretación
Suite `pytest`	166 pruebas aprobadas; 0 fallidas; 1 advertencia de ZIP duplicado.
Demostraciones desde IDE	Los ejemplos de MIPS se ejecutaron correctamente durante la revisión manual.
Revisión de código	Se identificaron módulos de generación, marcos, registros, bibliotecas y pruebas.
Entrega en GitHub	Repositorio identificado; los commits individuales deben revisarse en su historial.

9 Limitaciones y consideraciones

La documentación técnica declara que el generador TAC no soporta `try/catch`; tampoco se permite imprimir objetos o arreglos completos mediante `print`. Los enteros se manejan como valores de 32 bits con el comportamiento de desbordamiento de MIPS. Estas limitaciones deben explicarse durante la demostración y no confundirse con errores de ejecución imprevistos.

10 Conclusiones

El trabajo realizado cubre los tres aspectos principales de esta fase: secuencias de llamada y retorno, asignación de registros o uso de pila y generación de código MIPS ejecutable. La integración con el IDE y la batería de pruebas proporciona mecanismos reproducibles de validación. Las bibliotecas compiladas y la herencia entre archivos amplían el alcance funcional más allá del enunciado mínimo.

La ejecución de las pruebas automatizadas y los ejemplos desde el IDE ofrece una base concreta para la presentación. Como cierre de la entrega, corresponde mantener el repositorio actualizado, explicar las decisiones de implementación y demostrar el funcionamiento del código generado en MARS.

Referencias técnicas

1. *Proyecto Fase 3: Generación de Código*, Construcción de Compiladores, octubre de 2026 (enunciado proporcionado).
2. Repositorio Compiler Lab: `https://github.com/Emadlgg/compiler-toolkit`. Documentación técnica en `docs/mips.md`.
3. Repositorio del proyecto, `requirements.txt` y módulos `compiscript/mips/`.
4. Registro de ejecución local de pruebas compartido en la conversación: `166 passed, 1 warning`.